Automata

1.0

Generated by Doxygen 1.13.2

1 Hierarchical Index 1

1.1 Class Hierarchy . . . 1

2 Class Index 3

2.1 Class List . . . 3

3 File Index 5

3.1 File List . . . 5

4 Class Documentation 7

4.1 Automata Class Reference . . . 7

4.2 CFG< V_t, S_t > Class Template Reference . . . 7

4.3 DFA< Q_t, S_t > Class Template Reference . . . 8

4.4 epsilon Class Reference . . . 8

4.5 std::hash< std::list< T > > Struct Template Reference . . . 8

4.6 std::hash< std::pair< S, T > > Struct Template Reference . . . 9

4.7 std::hash< std::set< S > > Struct Template Reference . . . 9

4.8 std::hash< std::unordered_map< S, T, H > > Struct Template Reference . . . 9

4.9 std::hash< std::unordered_set< T > > Struct Template Reference . . . 9

4.10 std::hash< std::vector< T > > Struct Template Reference . . . 10

4.11 NFA< Q_t, S_t > Class Template Reference . . . 10

4.11.1 Detailed Description . . . 11

4.12 Parse_Tree Class Reference . . . 11

4.13 Parse_Tree_Node Class Reference . . . 11

4.14 PDA Class Reference . . . 12

4.15 RegEx Class Reference . . . 12

5 File Documentation 13

5.1 Automata.h . . . 13

5.2 CFG.h . . . 13

5.3 /home/student/Desktop/Project/Automata-Libary/include/DFA.h File Reference . . . 14

5.3.1 Detailed Description . . . 14

5.4 DFA.h . . . 15

5.5 /home/student/Desktop/Project/Automata-Libary/include/NFA.h File Reference . . . 18

5.5.1 Detailed Description . . . 18

5.6 NFA.h . . . 19

5.7 PDA.h . . . 23

5.8 Regex.h . . . 23

5.9 Regular_Language.h . . . 25

5.10 STL_util.h . . . 26

Index 35

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Automata . . . 7
CFG< V_t, S_t > . . . 7
    PDA . . . 12
DFA< Q_t, S_t > . . . 8
std::hash< std::list< T > > . . . 8
std::hash< std::pair< S, T > > . . . 9
std::hash< std::set< S > > . . . 9
std::hash< std::unordered_map< S, T, H > > . . . 9
std::hash< std::unordered_set< T > > . . . 9
std::hash< std::vector< T > > . . . 10
NFA< Q_t, S_t > . . . 10
Parse_Tree . . . 11
Parse_Tree_Node . . . 11
RegEx . . . 12
S_t
    epsilon . . . 8

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Automata . . . 7
CFG< V_t, S_t > . . . 7
DFA< Q_t, S_t > . . . 8
epsilon . . . 8
std::hash< std::list< T > > . . . 8
std::hash< std::pair< S, T > > . . . 9
std::hash< std::set< S > > . . . 9
std::hash< std::unordered_map< S, T, H > > . . . 9
std::hash< std::unordered_set< T > > . . . 9
std::hash< std::vector< T > > . . . 10
NFA< Q_t, S_t >
    Nondeterministic Finite-State Machine . . . 10
Parse_Tree . . . 11
Parse_Tree_Node . . . 11
PDA . . . 12
RegEx . . . 12

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

/home/student/Desktop/Project/Automata-Libary/include/Automata.h 13
/home/student/Desktop/Project/Automata-Libary/include/CFG.h 13
/home/student/Desktop/Project/Automata-Libary/include/DFA.h
DFA implementation 14
/home/student/Desktop/Project/Automata-Libary/include/NFA.h
NFA implementation 18
/home/student/Desktop/Project/Automata-Libary/include/PDA.h 23
/home/student/Desktop/Project/Automata-Libary/include/Regex.h 23
/home/student/Desktop/Project/Automata-Libary/include/Regular_Language.h 25
/home/student/Desktop/Project/Automata-Libary/include/STL_util.h 26

Chapter 4

Class Documentation

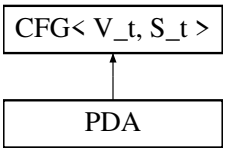
4.1 Automata Class Reference

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Automata-Libary/include/Automata.h

4.2 CFG< V_t, S_t > Class Template Reference

Inheritance diagram for CFG< V_t, S_t >:



Public Member Functions

- Setv **Epsilon_Productions** () const

Public Attributes

- Setv **V_**
- unordered_set< T_t > **sigma_**
- unordered_map< V_t, Bodies > **P_**
- V_t **S_**
- Setv **nullables_**

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Automata-Libary/include/CFG.h

4.3 DFA< Q_t, S_t > Class Template Reference

Public Member Functions

- **DFA** (const SetUs &sigma, const SetUq &Q, const SetUq &F, const Q_t &q0, const Mapt &delta)
- **DFA** (const DFA &M)
- bool **operator()** (const vector< S_t > &word, bool verbose=false)
- SuperSetUq **Moore_partition_** (const SuperSetUq &P) const
- SuperSetUq **Moore_partition** (const SetUq &S, const SuperSetUq &P)
- DFA< SetUq, S_t > **minimizeDFA** () const
- DFA< PairQQ, S_t > **intersection_** (const DFA &M) const
- DFA< PairQQ, S_t > **union_** (const DFA &M) const
- DFA & **compDFA** ()
- DFA< int, S_t > **baseDFA** () const

Public Attributes

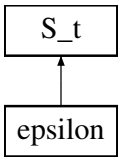
- Q_t **q0_**
- SetUq **Q_**
- SetUs **sigma_**
- Mapt **delta_**
- SetUq **F_**

The documentation for this class was generated from the following files:

- /home/student/Desktop/Project/Automata-Libary/include/DFA.h
- /home/student/Desktop/Project/Automata-Libary/src/DFA.cpp

4.4 epsilon Class Reference

Inheritance diagram for epsilon:



The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Automata-Libary/src/Regular_Language.cpp

4.5 std::hash< std::list< T > > Struct Template Reference

Public Member Functions

- size_t **operator()** (const std::list< T > &x) const

The documentation for this struct was generated from the following file:

- /home/student/Desktop/Project/Automata-Libary/include/STL_util.h

4.6 std::hash< std::pair< S, T > > Struct Template Reference

Public Member Functions

- size_t **operator()** (const std::pair< S, T > &x) const

The documentation for this struct was generated from the following file:

- /home/student/Desktop/Project/Automata-Libary/include/STL_util.h

4.7 std::hash< std::set< S > > Struct Template Reference

Public Member Functions

- size_t **operator()** (const std::set< S > &x) const

The documentation for this struct was generated from the following file:

- /home/student/Desktop/Project/Automata-Libary/include/STL_util.h

4.8 std::hash< std::unordered_map< S, T, H > > Struct Template Reference

Public Member Functions

- size_t **operator()** (const std::unordered_map< S, T, H > &x) const

The documentation for this struct was generated from the following file:

- /home/student/Desktop/Project/Automata-Libary/include/STL_util.h

4.9 std::hash< std::unordered_set< T > > Struct Template Reference

Public Member Functions

- size_t **operator()** (const std::unordered_set< T > &x) const

The documentation for this struct was generated from the following file:

- /home/student/Desktop/Project/Automata-Libary/include/STL_util.h

4.10 std::hash< std::vector< T > > Struct Template Reference

Public Member Functions

- size_t **operator()** (const std::vector< T > &x) const

The documentation for this struct was generated from the following file:

- /home/student/Desktop/Project/Automata-Libary/include/STL_util.h

4.11 NFA< Q_t, S_t > Class Template Reference

Nondeterministic Finite-State Machine.

`#include <NFA.h>`

Public Member Functions

- **NFA** (const SetUs &sigma, const SetUq &Q, const SetUq &F, const Q_t &q0, const Mapt &delta, const Map &epDt)

  An NFA contains the alphabets, the set of states, the Final states which is a subset of the set of states, the start state, the transition function and the epsilon transition function.
- **NFA** (const NFA &N)
- NFA **operator=** (const NFA &N)
- void **insert** (const S_t &a)
- int **size** () const

Word Processor

I implemented the word processing with a depth first graph search, i dont think i used the processing because the word processing is much slower than the dfa search.

- bool **operator()** (const vector< S_t > &word, bool verbose=false)

Union Operation

The union is connecting both NFA's to a start state with a epsilon

- NFA< int, S_t > **union_** (const NFA &N) const

Concatenate Operation

Concatenation is the connection of the final states of the first NFA to the start states of the second NFA with epsilons connections, there is an extra state that both are connected to

- NFA< int, S_t > **concatenate** (const NFA &N) const

Klene Star Operation

I implemented a KleneStar by connecting the final states to the start state and making the start state a final state.

- NFA< int, S_t > **KleneStar** () const

Base NFA

For most of the application of the NFA i needed to represent the states as integers to make it easier to add and to do operations, so this method is to convert the complex representation of states to an integer state with a starting point

- NFA< int, S_t > **baseNFA** (int start=0) const

Epsilon Closure

epsilon closure is to compute all the states that a set of states can reach with epsilon transitions

- SetUq **epsilon_closure** (const SetUq &S) const

Public Attributes

- Q_t **q0_**
- SetUq **Q_**
- SetUs **sigma_**
- Mapt **delta_**
- SetUq **F_**
- Map **epDt_**

4.11.1 Detailed Description

template<typename Q_t, typename S_t>
class NFA< Q_t, S_t >

Nondeterministic Finite-State Machine.

Represents a nondeterministic finite-state machine parameterized by its state type and input symbol type.

Template Parameters

Q_t	State type.
S_t	Input symbol type.

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Automata-Libary/include/NFA.h

4.12 Parse_Tree Class Reference

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Automata-Libary/include/CFG.h

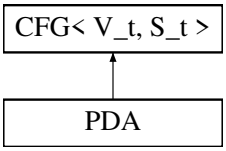
4.13 Parse_Tree_Node Class Reference

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Automata-Libary/include/CFG.h

4.14 PDA Class Reference

Inheritance diagram for PDA:



Additional Inherited Members

Public Member Functions inherited from CFG< V_t, S_t >

- Setv **Epsilon_Productions** () const

Public Attributes inherited from CFG< V_t, S_t >

- Setv **V_**
- unordered_set< T_t > **sigma_**
- unordered_map< V_t, Bodies > **P_**
- V_t **S_**
- Setv **nullables_**

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Automata-Libary/include/PDA.h

4.15 RegEx Class Reference

Public Member Functions

- **RegEx** (char c)
- **RegEx** (const string &x)
- **RegEx** (const NFA< int, char > &N)
- **RegEx** (const RegEx &r)
- RegEx & **operator=** (const RegEx &r)
- RegEx & **operator|=** (const RegEx &r)
- const RegEx **operator|** (const RegEx &r) const
- RegEx **operator+** () const
- RegEx **operator*** () const
- RegEx & **operator&=** (const RegEx &r)
- RegEx **operator&** (const RegEx &r) const
- DFAic **powerset_construction** () const

Public Attributes

- NFA< int, char > **N_**
- string **regex_**

The documentation for this class was generated from the following file:

- /home/student/Desktop/Project/Automata-Libary/include/Regex.h

Chapter 5

File Documentation

5.1 Automata.h

```
00001 #ifndef AUTOMATA_H
00002 #define AUTOMATA_H
00003
00004 #include "DFA.h"
00005 #include "NFA.h"
00006 #include "Regex.h"
00007 #include "CFG.h"
00008 #include "PDA.h"
00009
00010 class Automata
00011 {
00012 public:
00013 private:
00014 };
00015
00016 #endif
```

5.2 CFG.h

```
00001 #ifndef CFG_H
00002 #define CFG_H
00003
00004 template <typename V_t, typename S_t >
00005 class CFG
00006 {
00007     class Symbols: public V_t, T_t
00008     {
00009     public:
00010         Symbols(const V_t & v)
00011             : V_t(v), is_terminal(false)
00012         {}
00013         Symbols(const T_t & t)
00014             : T_t(t), is_terminal(true)
00015         {}
00016         const bool is_terminal;
00017
00018     };
00019     using Body   = vectors<Symbols>;
00020     using Bodies = unordered_set<Body>;
00021     using Setv   = unordered_set<V_t>;
00022 public:
00023
00024     Setv Epsilon_Productions() const;
00025
00026     Setv V_;
00027     unordered_set<T_t> sigma_;
00028     unordered_map<V_t, Bodies> P_;
00029     V_t S_;
00030     Setv nullables_;
00031 };
00032
00033 class Parse_Tree
00034 {
```

```
00035     Parse_Tree_Node root_;
00036 };
00037
00038 class Parse_Tree_Node
00039 {
00040     Token x;
00041     vector<Parse_Tree_Node> children_;
00042 };
00043
00044 #endif
```

5.3 /home/student/Desktop/Project/Automata-Libary/include/DFA.h File Reference

DFA implementation.

```
#include "STL_util.h"
```

Classes

- class DFA< Q_t, S_t >

Functions

- template<typename Q_t, typename S_t>
  void **walk** (const pair< Q_t, Q_t > &qi, const unordered_map< pair< Q_t, S_t >, Q_t > &delta0, const unordered_map< pair< Q_t, S_t >, Q_t > &delta1, unordered_map< pair< pair< Q_t, Q_t >, S_t >, pair< Q_t, Q_t > > &delta_, const unordered_set< S_t > &alphabet, unordered_set< pair< Q_t, Q_t > > &states)
- template<typename Q_t, typename S_t>
  DFA< int, S_t > **baseDFA** (int start) const
- template<typename S_t, typename Q_t>
  ostream & **operator**<< (ostream &cout, const DFA< S_t, Q_t > &M)

5.3.1 Detailed Description

DFA implementation.

Author

Oladayo Oyedeji

Date

11/01/2026

5.4 DFA.h

Go to the documentation of this file.

```
00001
00009 #ifndef DFA_H
00010 #define DFA_H
00011
00012 #include "STL_util.h"
00013
00014 template < typename Q_t, typename S_t >
00015 class DFA;
00016
00017 template < typename Q_t, typename S_t >
00018 void walk(const pair<Q_t, Q_t> & qi,
00019           const unordered_map< pair<Q_t, S_t>, Q_t> & delta0,
00020           const unordered_map< pair<Q_t, S_t>, Q_t> & delta1,
00021           unordered_map< pair< pair<Q_t, Q_t> , S_t>,
00022           pair<Q_t, Q_t> > & delta_, const unordered_set<S_t> & alphabet,
00023           unordered_set< pair<Q_t, Q_t> > & states);
00024
00025 template <typename Q_t, typename S_t>
00026 class DFA
00027 {
00028     using SetUq = unordered_set<Q_t>;
00029     using SetUs = unordered_set<S_t>;
00030     using SuperSetUq = unordered_set< SetUq >;
00031     using PairQS = pair<Q_t, S_t>;
00032     using PairQQ = pair<Q_t, Q_t>;
00033     using Mapt = unordered_map<PairQS, Q_t>;
00034 public:
00035     DFA(const SetUs & sigma, const SetUq & Q,
00036         const SetUq & F,     const Q_t   & q0,
00037         const Mapt  & delta);
00038     DFA(const DFA & M);
00039     bool operator()(const vector<S_t> & word, bool verbose=false);
00040     SuperSetUq Moore_partition_(const SuperSetUq & P) const;
00041     SuperSetUq Moore_partition(const SetUq & S, const SuperSetUq & P);
00042     DFA<SetUq, S_t> minimizeDFA() const;
00043     DFA<PairQQ, S_t> intersection_(const DFA & M) const;
00044     DFA<PairQQ, S_t> union_(       const DFA & M) const;
00045     DFA & compDFA();
00046     DFA<int, S_t> baseDFA() const;
00047
00048     Q_t   q0_;
00049     SetUq Q_;
00050     SetUs sigma_;
00051     Mapt  delta_;
00052     SetUq F_;
00053 };
00054
00055 template <typename Q_t, typename S_t>
00056 DFA<Q_t, S_t>::DFA(const SetUs & sigma, const SetUq & Q,
00057     const SetUq & F,     const Q_t   & q0,
00058     const Mapt  & delta)
00059     : q0_(q0), Q_(Q), sigma_(sigma), delta_(delta), F_(F)
00060 {}
00061
00062 template <typename Q_t, typename S_t>
00063 DFA<Q_t, S_t>::DFA(const DFA & M)
00064     : q0_(M.q0_), Q_(M.Q_), sigma_(M.sigma_), delta_(M.delta_), F_(M.F_)
00065 {}
00066
00067 template <typename Q_t, typename S_t>
00068 bool DFA<Q_t, S_t>::operator()(const vector<S_t> & word, bool verbose)
00069 {
00070     Q_t q = q0_;
00071     std::cout « word « std::endl;
00072
00073     for (int i = 0; i < word.size(); ++i)
00074     {
00075         q = delta_[{q,word[i]}];
00076
00077         if (verbose)
00078         {
00079             vector<S_t> v(word.begin() + i + 1, word.end());
00080             std::cout « std::pair<Q_t, std::vector<S_t»(q, v) « " " « word[i] « std::endl;
00081         }
00082     }
00083
00084     return is_member(q, F_);
00085 }
00086
00087 template <typename Q_t, typename S_t>
00088 typename DFA<Q_t, S_t>::SuperSetUq
00089 DFA<Q_t, S_t>::Moore_partition_(const SuperSetUq & P) const
```

```
00090 {
00091     SuperSetUq next;
00092     for (auto & block: P)
00093     {
00094         SuperSetUq peices = partition(block, P);
00095         next.insert(peices);
00096     }
00097     return next;
00098 }
00099
00100 template <typename Q_t, typename S_t>
00101 typename DFA<Q_t, S_t>::SuperSetUq
00102 DFA<Q_t, S_t>::Moore_partition(const SetUq & S, const SuperSetUq & P)
00103 {
00104     vector<SetUq> vecP(P.begin(), P.end());
00105     int i = 0;
00106
00107     unordered_map< unordered_set<int>, SetUq> P_k;
00108
00109     for (auto q: S)
00110     {
00111         unordered_set<int> index;
00112
00113         for (auto a: sigma_)
00114         {
00115             Q_t qf = delta_[{q, a}];
00116             for (int i = 0; i < vecP.size(); ++i)
00117             {
00118                 if (is_member(qf, vecP[i]))
00119                 {
00120                     index.insert(i);
00121                     break;
00122                 }
00123             }
00124         }
00125         P_k[index].insert(q);
00126     }
00127
00128     SuperSetUq result;
00129     for (auto & kv: P_k)
00130     {
00131         result.insert(kv.second);
00132     }
00133
00134     return result;
00135 }
00136
00137 template <typename Q_t, typename S_t>
00138 DFA<typename DFA<Q_t, S_t>::SetUq, S_t> DFA<Q_t, S_t>::minimizeDFA() const
00139 {
00140     SuperSetUq P;
00141     P.insert(Q_ - F_); P.insert(F_);
00142
00143     while (true)
00144     {
00145         SuperSetUq P_t = Moore_partition(P);
00146
00147         if (P_t == P)
00148             break;
00149
00150         P = P_t;
00151     }
00152
00153     SuperSetUq Q = P;
00154     SetUq q0;
00155
00156     // return DFA< SetUq, S_t>(sigma_, Q, F, q0, delta);
00157 }
00158
00159 template <typename Q_t, typename S_t>
00160 DFA<typename DFA<Q_t, S_t>::PairQQ, S_t>
00161 DFA<Q_t, S_t>::intersection_(const DFA & M) const
00162 {
00163     SetUs       sigma = sigma_ & M.sigma_;
00164     unordered_set<PairQQ> states;
00165     unordered_set<PairQQ> final_ = F_ * M.F_;
00166     PairQQ      start = PairQQ(q0_, M.q0_);
00167     unordered_map< pair< PairQQ, S_t >, PairQQ > delta;
00168
00169     walk<Q_t, S_t>(start, delta_, M.delta_, delta, sigma, states);
00170
00171     return DFA< PairQQ, S_t >(sigma, states, final_, start, delta);
00172 }
00173
00174 template <typename Q_t, typename S_t>
00175 DFA<typename DFA<Q_t, S_t>::PairQQ, S_t>
00176 DFA<Q_t, S_t>::union_(const DFA & M) const
```

```
00177 {
00178     SetUs       sigma = sigma_ & M.sigma_;
00179     unordered_set<PairQQ> states;
00180     unordered_set<PairQQ> final_ = (F_ * M.Q_) | (M.F_ * Q_);
00181     PairQQ      start = PairQQ(q0_, M.q0_);
00182     unordered_map< pair< PairQQ, S_t >, PairQQ > delta;
00183 
00184     walk<Q_t, S_t>(start, delta_, M.delta_, delta, sigma, states);
00185 
00186     return DFA< PairQQ, S_t >(sigma, states, final_, start, delta);
00187 }
00188 
00189 template <typename Q_t, typename S_t>
00190 DFA<Q_t, S_t> & DFA<Q_t, S_t>::compDFA()
00191 {
00192     F_ = Q_ - F_;
00193 
00194     return *this;
00195 }
00196 
00197 template <typename Q_t, typename S_t>
00198 DFA<int, S_t> baseDFA(int start) const
00199 {
00200     unordered_map<Q_t, int> mapper;
00201     SetUi Q;
00202     Q.insert(start);
00203     mapper[q0_] = start;
00204 
00205     SetUq Q_i = Q_;
00206     Q_i.erase(q0_);
00207 
00208     SetUi F;
00209     if (is_member(q0_, F_))
00210         F.insert(start);
00211 
00212     int i = start + 1;
00213     for (auto p: Q_i)
00214     {
00215         mapper[p] = i;
00216         Q.insert(i);
00217         if (is_member(p, F_))
00218         {
00219             F.insert(i);
00220         }
00221         i++;
00222     }
00223     unordered_map< pair<int, S_t>, int> delta;
00224 
00225     for (auto q: Q_)
00226     {
00227         int key = mapper[q];
00228         for (auto a: sigma_)
00229         {
00230             auto it = delta_.find({q, a});
00231             if (it != delta_.end())
00232             {
00233                 auto p = it->second;
00234                 int value = mapper[p];
00235                 delta[{key, a}].insert(value);
00236             }
00237         }
00238     }
00239 
00240     return DFA<int, S_t>(sigma_, Q, F, start, delta);
00241 }
00242 
00243 template < typename Q_t, typename S_t>
00244 void walk(const pair<Q_t, Q_t> & qi,
00245           const unordered_map< pair<Q_t, S_t>, Q_t> & delta0,
00246           const unordered_map< pair<Q_t, S_t>, Q_t> & delta1,
00247           unordered_map< pair< pair<Q_t, Q_t> , S_t>,
00248           pair<Q_t, Q_t> > & delta_, const unordered_set<S_t> & sigma,
00249           unordered_set< pair<Q_t, Q_t> > & states)
00250 {
00251     if (is_member(qi, states))
00252     {
00253         return;
00254     }
00255     else
00256     {
00257         states.insert(qi);
00258 
00259         Q_t a = qi.first;
00260         Q_t b = qi.second;
00261 
00262         for (auto p: sigma)
00263         {
```

```
00264             pair<Q_t, S_t> key0 =  pair<Q_t, S_t>(a, p);
00265             pair<Q_t, S_t> key1 =  pair<Q_t, S_t>(b, p);
00266
00267             Q_t value = delta0.at(key0);
00268
00269             pair<Q_t, Q_t> qi1 = pair<Q_t, Q_t>(delta0.at(key0), delta1.at(key1));
00270             delta_[{qi, p}] = qi1;
00271
00272             walk(qi1, delta0, delta1, delta_, sigma, states);
00273         }
00274     }
00275 }
00276
00277 template <typename S_t, typename Q_t >
00278 ostream & operator«(ostream & cout, const DFA< S_t, Q_t > & M)
00279 {
00280     cout « "M:" « endl;
00281     cout « "\tSigma: " « M.sigma_ « endl;
00282     cout « "\tStart State: " « M.q0_ « endl;
00283     cout « "\tSet of States: " « M.Q_ « endl;
00284     cout « "\tTransitions: " « M.delta_ « endl;
00285     cout « "\tFinal States: " « M.F_ « endl;
00286
00287     return cout;
00288 }
00289
00290 #endif
```

5.5 /home/student/Desktop/Project/Automata-Libary/include/NFA.h File Reference

NFA implementation.

```
#include "STL_util.h"
```

Classes

- class NFA< Q_t, S_t >

  Nondeterministic Finite-State Machine.

Functions

Epsilon Closure

epsilon closure is to compute all the states that a set of states can reach with epsilon transitions

- template<typename Q_t, typename S_t>
  ostream & **operator**<< (ostream &cout, const NFA< Q_t, S_t > &N)

5.5.1 Detailed Description

NFA implementation.

Author

Oladayo Oyedeji

Date

10/29/2026

NFA is different from a DFA in the sense that the transition is not deterministics hence the name

5.6 NFA.h

Go to the documentation of this file.

```
00001
00010 #ifndef NFA_H
00011 #define NFA_H
00012
00013 #include "STL_util.h"
00024 template <typename Q_t, typename S_t>
00025 class NFA
00026 {
00030     using SetUq = unordered_set<Q_t>;
00031     using SetUs = unordered_set<S_t>;
00032     using SuperSetUq = unordered_set< SetUq >;
00033     using PairQS = pair<Q_t, S_t>;
00034     using PairQQ = pair<Q_t, Q_t>;
00035     using PairS = pair<int, S_t>;
00036     using Mapt = unordered_map<PairQS, SetUq>;
00037     using SetUi = unordered_set<int>;
00038     using Mapi = unordered_map<PairS, SetUi>;
00039     using Map = unordered_map<Q_t, SetUq>;
00040     using MapUi = unordered_map<int, SetUi>;
00041 public:
00047     NFA(const SetUs & sigma, const SetUq & Q,
00048         const SetUq & F,     const Q_t   & q0,
00049         const Mapt  & delta, const Map   & epDt);
00050     NFA(const NFA & N);
00051     NFA()
00052         : q0_(0), Q_({0}), sigma_({}), F_({0})
00053     {}
00054
00055     NFA operator=(const NFA & N)
00056     {
00057         sigma_ = N.sigma_;
00058         Q_     = N.Q_;
00059         F_     = N.F_;
00060         q0_    = N.q0_;
00061         delta_ = N.delta_;
00062         epDt_  = N.epDt_;
00063
00064         return *this;
00065     }
00066     bool operator()(const vector<S_t> & word, bool verbose=false);
00067     NFA<int, S_t> union_(const NFA & N) const;
00068     NFA<int, S_t> concatenate(const NFA & N) const;
00069     NFA<int, S_t> KleneStar() const;
00070     NFA<int, S_t> baseNFA(int start=0) const;
00071     SetUq         epsilon_closure(const SetUq & S) const;
00072     void insert(const S_t & a)
00073     {
00074         int size = Q_.size();
00075         if (size == 0)
00076         {
00077             q0_ = size;
00078         }
00079         Q_.insert(q0_);
00080
00081     }
00082
00083     int size() const
00084     {
00085         return Q_.size();
00086     }
00087
00088     Q_t   q0_;
00089     SetUq Q_;
00090     SetUs sigma_;
00091     Mapt  delta_;
00092     SetUq F_;
00093     Map   epDt_;
00094
00095 };
00096
00097 template <typename Q_t, typename S_t>
00098 NFA<Q_t, S_t>::NFA(const SetUs & sigma, const SetUq & Q,
00099                    const SetUq & F,     const Q_t   & q0,
00100                    const Mapt  & delta, const Map   & epDt)
00101     : q0_(q0), Q_(Q), sigma_(sigma), delta_(delta), F_(F), epDt_(epDt)
00102 {}
00103
00104 template <typename Q_t, typename S_t>
00105 NFA<Q_t, S_t>::NFA(const NFA & N)
00106     : q0_(N.q0_), Q_(N.Q_), sigma_(N.sigma_), delta_(N.delta_), F_(N.F_), epDt_(N.epDt_)
00107 {}
00108
```

```
00114 template <typename Q_t, typename S_t>
00115 bool NFA<Q_t, S_t>::operator()(const vector<S_t> & word, bool verbose)
00116 {
00117     queue< pair<Q_t, int > > fringe;
00118     fringe.push({q0_, 0});
00119     int size = word.size();
00120
00121     SetUq closed_list;
00122     int i = 0;
00123     while (fringe.empty())
00124     {
00125         pair<Q_t, int> top = fringe.pop();
00126         Q_t q = top.first;
00127         int depth = top.second;
00128
00129         closed_list.insert(q);
00130
00131         if (depth >= size)
00132         {
00133             if (is_member(q, F_))
00134                 return true;
00135             continue;
00136         }
00137         auto it = delta_.find({q, word[i]});
00138
00139         if (it != delta_.end())
00140         {
00141             SetUq children_ = it->second;
00142             for (auto qi: children_)
00143             {
00144                 if (!is_member(qi, closed_list))
00145                 {
00146                     fringe.put({qi, depth+1});
00147                 }
00148             }
00149         }
00150         else
00151         {
00152             auto it = epDt_.find(q);
00153             for (auto p: it->second)
00154             {
00155                 if (it != delta_.end())
00156                 {
00157                     if (!is_member(p, closed_list))
00158                         fringe.put({p, depth});
00159                 }
00160             }
00161         }
00162     }
00163     return true;
00164 }
00165
00170 template <typename Q_t, typename S_t>
00171 NFA<int, S_t> NFA<Q_t, S_t>::union_(const NFA & N) const
00172 {
00173     NFA<int, S_t> N0 =   baseNFA(1);
00174     int size_ = size();
00175     NFA<int, S_t> N1 = N.baseNFA(size_ + 1);
00176     cout « N0 « endl;
00177     cout « N1 « endl;
00178     int q0 = 0;
00179     SetUi Q = N0.Q_ | N1.Q_;
00180     Q.insert(q0);
00181
00182     SetUs sigma = N0.sigma_ | N1.sigma_;
00183     Mapi delta = N0.delta_;
00184     delta.merge(N1.delta_);
00185
00186     MapUi epDt = N0.epDt_;
00187     epDt.merge(N1.epDt_);
00188
00189     SetUi F = N0.F_ | N1.F_;
00190
00191     // cout « size_ « ' ' « N1.q0_ « endl;
00192     // cout « N1 « endl;
00193
00194     epDt[q0] = {N0.q0_, N1.q0_};
00195     for (auto a: sigma)
00196     {
00197         // delta[{q0, a}] |= delta_[{  q0_, a}] | N.delta_[{N.q0_, a}];
00198         auto it = N0.delta_.find({N0.q0_, a});
00199         if (it != N0.delta_.end())
00200         {
00201              delta[{q0, a}] = delta[{q0, a}] | it->second;
00202         }
00203         it = N1.delta_.find({N1.q0_, a});
00204         //cout « it->second « endl;
```

```
00205         if (it != N1.delta_.end())
00206         {
00207             // cout « delta[{q0, a}] « ' ' « it->second« endl;
00208             delta[{q0, a}] = delta[{q0, a}] | it->second;
00209         }
00210     }
00211
00212     return NFA<int, S_t>(sigma, Q, F, q0, delta, epDt);
00213 }
00214
00221 template <typename Q_t, typename S_t>
00222 NFA<int, S_t> NFA<Q_t, S_t>::concatenate(const NFA & N) const
00223 {
00224     NFA<int, S_t> N0 =   baseNFA(0);
00225     NFA<int, S_t> N1 = N.baseNFA(size());
00226
00227     cout « "N0: " « N0 « endl;
00228     cout « "N1: " « N1 « endl;
00229
00230     SetUi Q = N0.Q_ | N1.Q_;
00231
00232     SetUs sigma = N0.sigma_ | N1.sigma_;
00233     // cout « sigma « endl;
00234     Mapi delta = N0.delta_;
00235     delta.merge(N1.delta_);
00236
00237     MapUi epDt = N0.epDt_;
00238     epDt.merge(N1.epDt_);
00239
00240     SetUi F = N1.F_;
00241
00242     for (auto q: N0.F_)
00243     {
00244         for (auto a: sigma)
00245         {
00246             // delta[{q,  a}] = delta[{q,a}] | N.delta_[{N.q0_, a}];
00247             auto it = N1.delta_.find({N1.q0_, a});
00248
00249             if (it != N1.delta_.end())
00250             {
00251                 delta[{q, a}] |= it->second;
00252             }
00253         }
00254         epDt[q].insert(N1.q0_);
00255     }
00256
00257     return NFA<int, S_t>(sigma, Q, F, N0.q0_, delta, epDt);
00258 }
00259
00265 template <typename Q_t, typename S_t>
00266 NFA<int, S_t> NFA<Q_t, S_t>::KleneStar() const
00267 {
00268     NFA<int, S_t> N =   baseNFA(1);
00269     cout « N « endl;
00270     int q0 = 0;
00271     SetUi Q = N.Q_;
00272     Q.insert(q0);
00273
00274     SetUs sigma = N.sigma_;
00275     Mapi delta = N.delta_;
00276
00277     MapUi epDt = N.epDt_;
00278     epDt[q0] = {N.q0_};
00279
00280     SetUi F = N.F_;
00281
00282     for (auto q: F)
00283     {
00284         for (auto a: sigma)
00285         {
00286             // delta[{q, a}] = delta[{q, a}] | delta_[{q0_, a}];
00287             auto it = N.delta_.find({N.q0_, a});
00288
00289             if (it != N.delta_.end())
00290             {
00291                 delta[{q, a}] |= it->second;
00292             }
00293         }
00294         epDt[q].insert(q0);
00295     }
00296     cout « F « ' ' « q0 « endl;
00297     F.insert(q0);
00298
00299     return NFA<int, S_t>(sigma, Q, F, q0, delta, epDt);
00300 }
00301
00308 template <typename Q_t, typename S_t>
```

```
00309 NFA<int, S_t> NFA<Q_t, S_t>::baseNFA(int start) const
00310 {
00311     unordered_map<Q_t, int> mapper;
00312     unordered_set<int> Q;
00313     Q.insert(start);
00314     mapper[q0_] = start;
00315
00316     SetUq Q_i = Q_;
00317     Q_i.erase(q0_);
00318
00319     unordered_set<int> F;
00320     if (is_member(q0_, F_))
00321         F.insert(start);
00322
00323     int i = start + 1;
00324     for (auto p: Q_i)
00325     {
00326         mapper[p] = i;
00327         Q.insert(i);
00328         if (is_member(p, F_))
00329         {
00330             F.insert(i);
00331         }
00332         i++;
00333     }
00334
00335     Mapi delta;
00336     unordered_map<int, unordered_set<int» epDt;
00337
00338     for (auto q: Q_)
00339     {
00340         int key = mapper[q];
00341         for (auto a: sigma_)
00342         {
00343
00344             auto it = delta_.find({q, a});
00345             if (it != delta_.end())
00346             {
00347                 for (auto p: it->second)
00348                 {
00349                     int value = mapper[p];
00350                     delta[{key,a}].insert(value);
00351                 }
00352             }
00353         }
00354         auto it = epDt_.find(q);
00355         if (it != epDt_.end())
00356         {
00357             for (auto p: it->second)
00358             {
00359                 int value = mapper[p];
00360                 epDt[key].insert(value);
00361             }
00362         }
00363     }
00364
00365     return NFA<int, S_t>(sigma_, Q, F, start, delta, epDt);
00366 }
00367
00373 template <typename Q_t, typename S_t >
00374 typename NFA<Q_t, S_t>::SetUq
00375 NFA<Q_t, S_t>::epsilon_closure(const SetUq & S) const
00376 {
00377     SetUq DONE;
00378     SetUq BOUNDARY = S;
00379     SetUq NEW;
00380
00381     while (true)
00382     {
00383         cout « "oh no" « endl;
00384         for (auto q: BOUNDARY)
00385         {
00386             auto it = epDt_.find(q);
00387             if (it != epDt_.end())
00388             {
00389                 SetUq s = it->second;
00390                 SetUq n = DONE | BOUNDARY;
00391                 NEW = s - n;
00392             }
00393         }
00394         DONE |= BOUNDARY;
00395
00396         if (NEW.empty())
00397             break;
00398         else
00399         {
00400             BOUNDARY |= NEW;
```

```
00401         }
00402     }
00403
00404     return DONE;
00405 }
00406
00407 template <typename Q_t, typename S_t >
00408 ostream & operator«(ostream & cout, const NFA< Q_t, S_t > & N)
00409 {
00410     cout « "N:" « endl;
00411     cout « "\tSigma: " « N.sigma_ « endl;
00412     cout « "\tStart State: " « N.q0_ « endl;
00413     cout « "\tSet of States: " « N.Q_ « endl;
00414     cout « "\tTransitions: " « N.delta_ « endl;
00415     cout « "\tFinal States: " « N.F_ « endl;
00416     cout « "\tEpsilon transition: " « N.epDt_ « endl;
00417
00418     return cout;
00419 }
00420
00421 #endif
```

5.7 PDA.h

```
00001 #ifndef PDA_H
00002 #define PDA_H
00003
00004 class PDA: public CFG
00005 {
00006     bool operator()(const std::vector<S_t>&);
00007     CYK();
00008     SRP();
00009 };
00010
00011 #endif
```

5.8 Regex.h

```
00001 #ifndef REGEX_H
00002 #define REGEX_H
00003
00004 #include "NFA.h"
00005
00006 typedef unordered_map<int, unordered_set<int» Mapi;
00007 typedef unordered_map<pair<int,char>, unordered_set<int» Mapc;
00008 typedef DFA<unordered_set<int>, char> DFAic;
00009 typedef DFA<int, char> DFAi;
00010 typedef unordered_set<int> Seti;
00011 typedef unordered_set< Seti > SuperSeti;
00012
00013
00014 class RegEx
00015 {
00016 public:
00017     RegEx(char c)
00018         : N_({c}, {0, 1}, {1}, 0,
00019              Mapc{ {0, c}, {1} } },   // 0 --c--> 1
00020              Mapi{}), regex_()
00021     {
00022         // cout « N_ « endl;
00023         regex_ += c;
00024     }
00025     RegEx(const string & x)
00026     {
00027         if (x.size() == 1)
00028         {
00029             *this = RegEx(x[0]);
00030             return;
00031         }
00032         cout « N_ « endl;
00033         for (int i = 0; i < x.size(); ++i)
00034         {
00035             *this &= RegEx(x[i]);
00036         }
00037     }
00038     RegEx(const NFA<int, char> & N)
00039         : N_(N)
00040     {}
```

```
00041     RegEx(const RegEx & r)
00042         : N_(r.N_)
00043     {}
00044     RegEx & operator=(const RegEx & r)
00045     {
00046         N_ = r.N_;
00047         regex_ = r.regex_;
00048 
00049         return *this;
00050     }
00051     // union
00052     RegEx & operator|=(const RegEx & r)
00053     {
00054         N_ = N_.union_(r.N_);
00055         return *this;
00056     }
00057     const RegEx operator|(const RegEx & r) const
00058     {
00059         RegEx R(*this);
00060         R |= r;
00061         return R;
00062     }
00063     // klene star and klene star - {e}
00064     RegEx operator+() const
00065     {
00066         NFA<int, char> N = N_.KleneStar();
00067         N.F_.erase(N_.q0_);
00068 
00069         return RegEx(N);
00070     }
00071     RegEx operator*() const
00072     {
00073         return RegEx(N_.KleneStar());
00074     }
00075     // concatenation
00076     RegEx & operator&=(const RegEx & r)
00077     {
00078         N_ = N_.concatenate(r.N_);
00079         regex_ += r.regex_;
00080         return *this;
00081     }
00082 
00083     RegEx operator&(const RegEx & r) const
00084     {
00085         RegEx R(*this);
00086         R &= r;
00087         return R;
00088     }
00089 
00090     DFAic powerset_construction() const
00091     {
00092         Seti q0 = N_.epsilon_closure({N_.q0_});
00093         int i = 0;
00094         cout « i « " : " « q0 « endl;
00095         // int x;
00096         // cin » x;
00097         SuperSeti Q;
00098         SuperSeti F;
00099         unordered_map<pair<Seti, char>, Seti> delta;
00100 
00101         stack<Seti> fringe;
00102         fringe.push(q0);
00103 
00104         while (!fringe.empty())
00105         {
00106             Seti top = fringe.top();
00107             fringe.pop();
00108             Q.insert(top);
00109 
00110             Seti intersect = top & N_.F_;
00111             F.insert(intersect);
00112 
00113             for (auto a: N_.sigma_)
00114             {
00115                 Seti next;
00116                 for (auto q: top)
00117                 {
00118                     auto it = N_.delta_.find({q, a});
00119                     if (it != N_.delta_.end())
00120                         next |= it->second;
00121                 }
00122 
00123                 next = N_.epsilon_closure(next);
00124                 cout « i « " : " « next « endl;
00125                 // int x;
00126                 // cin » x;
00127                 delta[{top, a}] = next;
```

```
00128
00129                 if (!is_member(next, Q))
00130                 {
00131                     fringe.push(next);
00132                 }
00133             }
00134         }
00135
00136         DFAic M(N_.sigma_, Q, F, q0, delta);
00137
00138         // cout « M.minimizeDFA() « endl;
00139
00140         return M;
00141     }
00142
00143     NFA<int, char> N_;
00144     string regex_;
00145
00146 };
00147
00148
00149 ostream & operator«(ostream & cout, const RegEx & r)
00150 {
00151     cout « "Regular expression: " « r.regex_ « endl;
00152     cout « r.N_;
00153     return cout;
00154 }
00155
00156 #endif
```

5.9 Regular_Language.h

```
00001 #ifndef REGULAR_LANGUAGE_H
00002 #define REGULAR_LANGUAGE_H
00003
00004 #include "DFA.h"
00005 #include "NFA.h"
00006
00007
00008 template <typename Q_t, typename S_t>
00009 std::set<Q_t> epsilon_closure(const std::set<Q_t>& start) const
00010 {
00011     std::set<Q_t> done;
00012     std::set<Q_t> boundary = start;
00013
00014     while (!boundary.empty())
00015     {
00016         std::set<Q_t> next;
00017
00018         for (const auto& q : boundary)
00019         {
00020             done.insert(q);
00021
00022             auto it = delta.find({q, ""});
00023             if (it == delta.end())
00024                 continue;
00025
00026             for (const auto& p : it->second)
00027             {
00028                 if (done.count(p) == 0 && boundary.count(p) == 0)
00029                     next.insert(p);
00030             }
00031         }
00032
00033         boundary = std::move(next);
00034     }
00035
00036     return done;
00037 }
00038
00039 template <typename Q_t, typename S_t>
00040 std::set<Q_t> epsilon_closure(const Q_t& q) const
00041 {
00042     return epsilon_closure(std::set<Q_t>{q});
00043 }
00044
00045 template <typename Q_t, typename S_t>
00046 DFA<std::set<Q_t>> powerset_construction() const
00047 {
00048     using S = std::set<Q_t>;
00049
00050     const S start = epsilon_closure(q0_);
00051
```

```
00052     std::set<S> states;
00053     std::set<S> accepting;
00054     std::map<std::pair<S, std::string>, S> transitions;
00055 
00056     std::stack<S> fringe;
00057     fringe.push(start);
00058     states.insert(start);
00059 
00060     while (!fringe.empty())
00061     {
00062         S top = fringe.top();
00063         fringe.pop();
00064 
00065         // accepting if it contains any NFA accepting state
00066         for (const auto& q : top)
00067         {
00068             if (F_.count(q))
00069             {
00070                 accepting.insert(top);
00071                 break;
00072             }
00073         }
00074 
00075         for (const auto& a : alphabet_)
00076         {
00077             S moved;
00078             for (const auto& q : top)
00079             {
00080                 auto it = delta.find({q, a});
00081                 if (it != delta.end())
00082                     moved.insert(it->second.begin(), it->second.end());
00083             }
00084 
00085             S next = epsilon_closure(moved);
00086             transitions[{top, a}] = next;
00087 
00088             if (states.insert(next).second)   // true if newly discovered
00089                 fringe.push(next);
00090         }
00091     }
00092 
00093     return DFA<S>(states, alphabet_, transitions, start, accepting);
00094 }
00095 
00096 
00097 #endif
```

5.10 STL_util.h

```
00001 #ifndef STL_UTIL_H
00002 #define STL_UTIL_H
00003 
00004 #include <iostream>
00005 #include <vector>
00006 #include <string>
00007 #include <list>
00008 #include <unordered_set>
00009 #include <unordered_map>
00010 #include <set>
00011 #include <stack>
00012 #include <queue>
00013 #include <algorithm>
00014 
00015 using namespace std;
00016 
00017 //================================================================================
00018 // Prototypes
00019 //
00020 // * std::string to_string(T) where T = int, double, char, bool,
00021 // const std::string &
00022 // const std::pair< X, Y > &
00023 // const std::vector< X > &
00024 // const std::list< X > & ... TODO
00025 // const std::unordered_map< X, Y > &
00026 // const std::unordered_set< X > &
00027 //
00028 // * std::hash< T > where T = int, double, char, bool,
00029 // const std::string &
00030 // const std::pair< X, Y > &
00031 // const std::vector< X > &
00032 // const std::list< X > & ... TODO
00033 // const std::unordered_map< X, Y > &
00034 // const std::unordered_set< X > &
```

```
00035 //
00036 // * set membership
00037 // If C is a container and e is an element, use one of the following for
00038 // membership check:
00039 // has< T, S > (C, e)
00040 // is_member< S, T >(e, C) where T is a container of S-values and
00041 // T::find(const S &) is defined
00042 // * set union
00043 // If X, Y, Z are containers of type std::set< T >, std::unordered_set< T >
00044 // X |= Y // X = X union with Y
00045 // Z = X | Y // Z = X union with Y
00046 // (See also std::set_union)
00047 //
00048 // * set intersection
00049 // If X, Y, Z are containers of type std::set< T >, std::unordered_set< T >
00050 // X &= Y
00051 // Z = X & Y
00052 // (See also C++ std::set_intersection)
00053 //
00054 // * set difference
00055 // If X, Y, Z are containers of type std::set< T >, std::unordered_set< T >
00056 // X -= Y
00057 // Z = X - Y
00058 // (See also C++ std::set_difference)
00059 //
00060 // * cross product
00061 // If X, Y, Z are containers of type std::set< T >, std::unordered_set< T >
00062 // Z = crossprod(X, Y)
00063 //
00064 // * powerset
00065 // If X, Z are containers of type of std::set< T >, std::unordered_set< T >
00066 // Z = powset(X)
00067 //
00068 //============================================================================
00069
00070 //----------------------------------------------------------------------------
00071 // Prototypes of std::to_string extensions
00072 //----------------------------------------------------------------------------
00073 namespace std
00074 {
00075     std::string to_string(char c);
00076
00077     std::string to_string(const std::string & x);
00078
00079     template < typename S, typename T >
00080     std::string to_string(const std::pair< S, T > & x);
00081
00082     template < typename T >
00083     std::string container_to_string_(const T & x,
00084                                      const std::string & begin,
00085                                      const std::string & end);
00086
00087     template < typename T >
00088     std::string to_string(const std::vector< T > & x);
00089
00090     template < typename T >
00091     std::string to_string(const std::list< T > & x);
00092
00093     template < typename T >
00094     std::string to_string(const std::unordered_set< T > & x);
00095
00096     template < typename S, typename T, typename LESS >
00097     std::string to_string(const std::unordered_map< S, T > & x,
00098                           const LESS & less);
00099
00100     template < typename S, typename T >
00101     std::string to_string(const std::unordered_map< S, T > & x);
00102
00103     template < typename T, typename LESS >
00104     std::string to_string(const std::unordered_set< T > & x,
00105                           const LESS & less);
00106     template < typename T, typename LESS >
00107     std::string to_string(const std::unordered_set< T > & x);
00108
00109     template <typename T >
00110     std::string to_string(const std::set< T > & x);
00111
00112     template < typename T, typename LESS >
00113     std::string to_string(const std::set< T > & x, const LESS & less);
00114 };
00115
00116 //----------------------------------------------------------------------------
00117 // Prototypes of std::hash extensions
00118 //----------------------------------------------------------------------------
00119 namespace std
00120 {
00121     template < typename S, typename T >
```

```
00122     struct hash< std::pair< S, T > >;
00123
00124     template < typename T >
00125     struct hash< std::vector< T > >;
00126
00127     template < typename T >
00128     struct hash< std::list< T > >;
00129
00130     template < typename T >
00131     struct hash< std::unordered_set< T > >;
00132
00133     template < typename S, typename T, typename H >
00134     struct hash< std::unordered_map< S, T, H > >;
00135
00136     template < typename S >
00137     struct hash< std::set< S > >;
00138 }
00139
00140 //-----------------------------------------------------------------------------
00141 // Prototypes of operator«
00142 //-----------------------------------------------------------------------------
00143 template < typename T0, typename T1 >
00144 std::ostream & operator«(std::ostream & cout, const std::pair< T0, T1 > & x);
00145
00146 template < typename T >
00147 std::ostream & operator«(std::ostream & cout, const std::vector< T > & s);
00148
00149 template < typename T >
00150 std::ostream & operator«(std::ostream & cout, const std::list< T > & s);
00151
00152 template < typename T >
00153 std::ostream & operator«(std::ostream & cout,
00154                          const std::unordered_set< T > & s);
00155
00156 template < typename S, typename T >
00157 std::ostream & operator«(std::ostream & cout,
00158                          const std::unordered_map< S, T > & s);
00159
00160 template < typename T >
00161 std::ostream & operator«(std::ostream & cout, const std::set< T > & s);
00162
00163 //-----------------------------------------------------------------------------
00164 // Prototypes of membership
00165 //-----------------------------------------------------------------------------
00166 template < typename S, typename T >
00167 bool is_member(const S & e, const T & X);
00168
00169 template < typename S, typename T >
00170 bool has(const S & X, const T & e);
00171
00172 //-----------------------------------------------------------------------------
00173 // Prototypes of set union
00174 //-----------------------------------------------------------------------------
00175 template < typename T >
00176 void set_union(T & x, const T & y);
00177
00178 template < typename T >
00179 std::set< T > & operator|=(std::set< T > & x, const std::set< T > & y);
00180
00181 template < typename T >
00182 std::set< T > operator|(const std::set< T > & x, const std::set< T > & y);
00183
00184 template < typename T >
00185 std::unordered_set< T > & operator|=(std::unordered_set< T > & x,
00186                                      const std::unordered_set< T > & y);
00187
00188 template < typename T >
00189 std::unordered_set< T > operator|(const std::unordered_set< T > & x,
00190                                   const std::unordered_set< T > & y);
00191
00192 //-----------------------------------------------------------------------------
00193 // Prototypes of set intersection
00194 //-----------------------------------------------------------------------------
00195 template < typename T >
00196 void set_intersection_(T & xs, const T & ys);
00197
00198 template < typename T >
00199 std::set< T > & operator&=(std::set< T > & xs, const std::set< T > & ys);
00200
00201 template < typename T >
00202 std::set< T > operator&(const std::set< T > & x, const std::set< T > & y);
00203
00204 template < typename T >
00205 std::unordered_set< T > operator&=(std::unordered_set< T > & xs,
00206                                    const std::unordered_set< T > & ys);
00207
00208 template < typename T >
```

```
00209 std::unordered_set< T > operator&(const std::unordered_set< T > & x,
00210                                   const std::unordered_set< T > & y);
00211
00212 //-----------------------------------------------------------------------------
00213 // Prototypes for set difference
00214 //-----------------------------------------------------------------------------
00215 template < typename T0, typename T1 >
00216 void set_difference_(T0 & X, const T1 & Y);
00217
00218 template < typename T >
00219 const std::unordered_set< T > & operator-=(std::unordered_set< T > & X,
00220                                            const std::unordered_set< T > & Y);
00221
00222 template < typename T >
00223 std::unordered_set< T > operator-(const std::unordered_set< T > & X,
00224                                   const std::unordered_set< T > & Y);
00225
00226 template < typename T >
00227 const std::set< T > & operator-=(std::set< T > & X, const std::set< T > & Y);
00228
00229 template < typename T >
00230 std::set< T > operator-(const std::set< T > & X, const std::set< T > & Y);
00231
00232 //-----------------------------------------------------------------------------
00233 // Prototypes of cross product
00234 //-----------------------------------------------------------------------------
00235 template < typename T0, typename T1, typename T2 >
00236 void crossprod_(T0 & ret, const T1 & xs, const T2 & ys);
00237
00238 template < typename T0, typename T1 >
00239 std::set< std::pair< T0, T1 > > crossprod(const std::set< T0 > & xs,
00240                                           const std::set< T1 > & ys);
00241
00242 template < typename T0, typename T1 >
00243 std::unordered_set< std::pair< T0, T1 > > crossprod(
00244     const std::unordered_set< T0 > & xs,
00245     const std::unordered_set< T1 > & ys);
00246
00247 template < typename T0, typename T1 >
00248 std::set< std::pair< T0, T1 > > operator*(const std::set< T0 > & xs,
00249                                           const std::set< T1 > & ys);
00250
00251 template < typename T0, typename T1 >
00252 std::unordered_set< std::pair< T0, T1 > > operator*(const std::unordered_set< T0 > & xs,
00253                                           const std::unordered_set< T1 > & ys);
00254
00255 //-----------------------------------------------------------------------------
00256 // Prototypes of powerset
00257 //-----------------------------------------------------------------------------
00258 template < typename T0, typename T1 >
00259 T0 powset_(const T1 & X);
00260
00261 template < typename T >
00262 std::set< std::set< T > > powset(const std::set< T > & X);
00263
00264 template < typename T >
00265 std::unordered_set< std::unordered_set< T > > powset(
00266     const std::unordered_set< T > & X);
00267
00268 //=============================================================================
00269 // Implementations
00270 //=============================================================================
00271 //-----------------------------------------------------------------------------
00272 // Implementation of std::to_string extensions
00273 //-----------------------------------------------------------------------------
00274 namespace std
00275 {
00276     inline
00277     std::string to_string(char c)
00278     {
00279         return {c};
00280     }
00281
00282     inline
00283     std::string to_string(const std::string & x)
00284     {
00285         return x;
00286     }
00287
00288     // template <typename T>
00289     // std::string to_string(const T & x)
00290     // {
00291
00292     // }
00293
00294     template <typename S, typename T>
00295     std::string to_string(const std::pair< S, T > & x)
```

```
00296     {
00297         return std::string("(") +
00298             to_string(x.first) + ", " + to_string(x.second) +
00299             ")";
00300     }
00301
00302     template < typename T >
00303     std::string container_to_string_(const T & x,
00304                                      const std::string & begin,
00305                                      const std::string & end)
00306     {
00307         std::string ret = begin;
00308         std::string delim = "";
00309         for (auto p: x)
00310         {
00311             ret += delim;
00312             ret += to_string(p);
00313             delim = ", ";
00314         }
00315         ret += end;
00316
00317         return ret;
00318     }
00319
00320     template < typename T >
00321     std::string to_string(const std::vector< T > & x)
00322     {
00323         return container_to_string_(x, "[", "]");
00324     }
00325
00326     // template < typename S, typename T, typename LESS >
00327     // std::string to_string(const std::unordered_map< S, T > & x,
00328     //                       const LESS & less)
00329     // {
00330
00331     // }
00332
00333     template < typename S, typename T >
00334     std::string to_string(const std::unordered_map< S, T > & x)
00335     {
00336         return container_to_string_(x, "{", "}");
00337     }
00338
00339     // template < typename T, typename LESS >
00340     // std::string to_string(const std::unordered_set< T > & x,
00341     //                       const LESS & less)
00342     // {
00343     //     return container_to_string_(x, "{", "}");
00344     // }
00345
00346     template < typename T >
00347     std::string to_string(const std::unordered_set< T > & x)
00348     {
00349         return container_to_string_(x, "{", "}");
00350     }
00351
00352     template <typename T >
00353     std::string to_string(const std::set< T > & x)
00354     {
00355         return container_to_string_(x, "{", "}");
00356     }
00357
00358     template < typename T, typename LESS >
00359     std::string to_string(const std::set< T > & x, const LESS & less)
00360     {
00361         return container_to_string_(x, "{", "}");
00362     }
00363 }
00364
00365 //-----------------------------------------------------------------------------
00366 // Implementations of extensions to std::hash
00367 //-----------------------------------------------------------------------------
00368 namespace std
00369 {
00370     template < typename S, typename T >
00371     struct hash< std::pair< S, T > >
00372     {
00373         size_t operator()(const std::pair< S, T > & x) const
00374         {
00375             std::hash< std::string > hasher;
00376             return hasher(std::to_string(x));
00377         }
00378     };
00379
00380     template < typename T >
00381     struct hash< std::vector< T > >
00382     {
```

```
00383         size_t operator()(const std::vector< T > & x) const
00384         {
00385             std::hash< std::string > hasher;
00386             return hasher(std::to_string(x));
00387         }
00388     };
00389
00390     template < typename T >
00391     struct hash< std::list< T > >
00392     {
00393         size_t operator()(const std::list< T > & x) const
00394         {
00395             std::hash< std::string > hasher;
00396             return hasher(std::to_string(x));
00397         }
00398     };
00399
00400     template < typename T >
00401     struct hash< std::unordered_set< T > >
00402     {
00403         size_t operator()(const std::unordered_set< T > & x) const
00404         {
00405             std::hash< std::string > hasher;
00406             return hasher(std::to_string(x));
00407         }
00408     };
00409
00410     template < typename S, typename T, typename H >
00411     struct hash< std::unordered_map< S, T, H > >
00412     {
00413         size_t operator()(const std::unordered_map< S, T, H > & x) const
00414         {
00415             std::hash< std::string > hasher;
00416             return hasher(std::to_string(x));
00417         }
00418     };
00419
00420     template < typename S >
00421     struct hash< std::set< S > >
00422     {
00423         size_t operator()(const std::set< S > & x) const
00424         {
00425             std::hash< std::string > hasher;
00426             return hasher(std::to_string(x));
00427         }
00428     };
00429 }
00430
00431 //--------------------------------------------------------------------------
00432 // Implementation of memberships
00433 //--------------------------------------------------------------------------
00434 template < typename S, typename T >
00435 bool is_member(const S & e,
00436                const T & X)
00437 {
00438     return X.find(e) != X.end();
00439 }
00440
00441 //--------------------------------------------------------------------------
00442 // Implementations of set difference
00443 //--------------------------------------------------------------------------
00444 template < typename T0, typename T1 >
00445 void set_difference_(T0 & X, const T1 & Y)
00446 {
00447     for (auto p: Y)
00448     {
00449         if (is_member(p, X))
00450             X.erase(p);
00451     }
00452 }
00453
00454 template < typename T >
00455 const std::unordered_set< T > & operator-=(std::unordered_set< T > & X,
00456                                            const std::unordered_set< T > & Y)
00457 {
00458     set_difference_(X, Y);
00459     return X;
00460 }
00461
00462 template < typename T >
00463 std::unordered_set< T > operator-(const std::unordered_set< T > & X,
00464                                   const std::unordered_set< T > & Y)
00465 {
00466     unordered_set<T> diff = X;
00467     diff -= Y;
00468     return diff;
00469 }
```

```
00470
00471 template < typename T >
00472 const std::set< T > & operator-=(std::set< T > & X, const std::set< T > & Y)
00473 {
00474     set_difference_(X, Y);
00475     return X;
00476 }
00477
00478 template < typename T >
00479 std::set< T > operator-(const std::set< T > & X, const std::set< T > & Y)
00480 {
00481     std::set<T> diff = X;
00482     diff -= Y;
00483     return diff;
00484 }
00485
00486 //-----------------------------------------------------------------------------
00487 // Implementations of set union
00488 //-----------------------------------------------------------------------------
00489 template < typename T >
00490 void set_union(T & x, const T & y)
00491 {
00492     for (auto p: y)
00493     {
00494         x.insert(p);
00495     }
00496 }
00497 template < typename T >
00498 std::set< T > & operator|=(std::set< T > & x, const std::set< T > & y)
00499 {
00500     set_union(x, y);
00501     return x;
00502 }
00503
00504 template < typename T >
00505 std::set< T > operator|(const std::set< T > & x, const std::set< T > & y)
00506 {
00507     std::set<T> union_ = x;
00508     union_ |= y;
00509     return union_;
00510 }
00511
00512 template < typename T >
00513 std::unordered_set< T > & operator|=(std::unordered_set< T > & x,
00514                                      const std::unordered_set< T > & y)
00515 {
00516     set_union(x, y);
00517     return x;
00518 }
00519
00520 template < typename T >
00521 std::unordered_set< T > operator|(const std::unordered_set< T > & x,
00522                                   const std::unordered_set< T > & y)
00523 {
00524     std::unordered_set<T> union_ = x;
00525     union_ |= y;
00526     return union_;
00527 }
00528
00529 //-----------------------------------------------------------------------------
00530 // Implementations of set intersection
00531 //-----------------------------------------------------------------------------
00532 template < typename T >
00533 void set_intersection_(T & xs, const T & ys)
00534 {
00535     T x = xs;
00536     xs.clear();
00537     for (auto p: x)
00538     {
00539         if (is_member(p, ys))
00540             xs.insert(p);
00541     }
00542 }
00543
00544 template < typename T >
00545 std::set< T > & operator&=(std::set< T > & xs, const std::set< T > & ys)
00546 {
00547     set_intersection_(xs, ys);
00548     return xs;
00549 }
00550
00551 template < typename T >
00552 std::set< T > operator&(const std::set< T > & xs, const std::set< T > & ys)
00553 {
00554     std::set<T> intersect_ = xs;
00555     intersect_ &= ys;
00556     return intersect_;
```

```
00557 }
00558
00559 template < typename T >
00560 std::unordered_set< T > operator&=(std::unordered_set< T > & xs,
00561                                    const std::unordered_set< T > & ys)
00562 {
00563     set_intersection_(xs, ys);
00564     return xs;
00565 }
00566
00567 template < typename T >
00568 std::unordered_set< T > operator&(const std::unordered_set< T > & xs,
00569                                   const std::unordered_set< T > & ys)
00570 {
00571     std::unordered_set<T> intersect_ = xs;
00572     intersect_ &= ys;
00573     return intersect_;
00574 }
00575
00576 //-----------------------------------------------------------------------------
00577 // Implementations of cross product
00578 //-----------------------------------------------------------------------------
00579 template < typename T0, typename T1, typename T2 >
00580 void crossprod_(T0 & ret, const T1 & xs, const T2 & ys)
00581 {
00582     for (auto p: xs)
00583     {
00584         for (auto q: ys)
00585         {
00586             ret.insert(std::make_pair(p, q));
00587         }
00588     }
00589 }
00590
00591 template < typename T0, typename T1 >
00592 std::set< std::pair< T0, T1 > > crossprod(const std::set< T0 > & xs,
00593                                           const std::set< T1 > & ys)
00594 {
00595     std::set< std::pair< T0, T1 > > cP;
00596     crossprod_(cP, xs, ys);
00597
00598     return cP;
00599 }
00600
00601 template < typename T0, typename T1 >
00602 std::unordered_set< std::pair< T0, T1 > > crossprod(
00603     const std::unordered_set< T0 > & xs,
00604     const std::unordered_set< T1 > & ys)
00605 {
00606     std::unordered_set< std::pair< T0, T1 > > cP;
00607     crossprod_(cP, xs, ys);
00608
00609     return cP;
00610 }
00611
00612 template < typename T0, typename T1 >
00613 std::set< std::pair< T0, T1 > > operator*(const std::set< T0 > & xs,
00614                                           const std::set< T1 > & ys)
00615 {
00616     return crossprod(xs, ys);
00617 }
00618
00619 template < typename T0, typename T1 >
00620 std::unordered_set< std::pair< T0, T1 > > operator*(const std::unordered_set< T0 > & xs,
00621                                                     const std::unordered_set< T1 > & ys)
00622 {
00623     return crossprod(xs, ys);
00624 }
00625 //-----------------------------------------------------------------------------
00626 // Implementations of powerset
00627 //-----------------------------------------------------------------------------
00628 template < typename T0, typename T1 >
00629 T0 powset_(const T1 & X)
00630 {
00631     T0 ret;
00632     std::vector<typename T1::value_type> elements(X.begin(), X.end());
00633
00634     int n = elements.size();
00635
00636     for (int mask = 0; mask < (1 << n); ++mask)
00637     {
00638         T1 subset;
00639
00640         for (int i = 0; i < n; ++i)
00641         {
00642             if (mask & (1 << i))
00643                 subset.insert(elements[i]);
```

```
00644         }
00645
00646         ret.insert(subset);
00647     }
00648
00649     return ret;
00650 }
00651
00652 template < typename T >
00653 std::set< std::set< T > > powset(const std::set< T > & X)
00654 {
00655     return powset_(X);
00656 }
00657
00658 template < typename T >
00659 std::unordered_set< std::unordered_set< T > > powset(
00660     const std::unordered_set< T > & X)
00661 {
00662     return powset_(X);
00663 }
00664
00665 //-----------------------------------------------------------------------------
00666 // Implementations of operator«
00667 //-----------------------------------------------------------------------------
00668 template < typename T0, typename T1 >
00669 std::ostream & operator«(std::ostream & cout, const std::pair< T0, T1 > & x)
00670 {
00671     cout « std::to_string(x);
00672     return cout;
00673 }
00674
00675 template < typename T >
00676 std::ostream & operator«(std::ostream & cout, const std::vector< T > & s)
00677 {
00678     cout « to_string(s);
00679     return cout;
00680 }
00681
00682 // template < typename T >
00683 // std::ostream & operator«(std::ostream & cout, const std::list< T > & s)
00684 // {
00685 //     cout « to_string(s);
00686 //     return cout;
00687 // }
00688
00689 template < typename T >
00690 std::ostream & operator«(std::ostream & cout,
00691                          const std::unordered_set< T > & s)
00692 {
00693     cout « to_string(s);
00694     return cout;
00695 }
00696
00697 template < typename S, typename T >
00698 std::ostream & operator«(std::ostream & cout,
00699                          const std::unordered_map< S, T > & s)
00700 {
00701     cout « to_string(s);
00702     return cout;
00703 }
00704
00705 template < typename T >
00706 std::ostream & operator«(std::ostream & cout, const std::set< T > & s)
00707 {
00708     cout « to_string(s);
00709     return cout;
00710 }
00711
00712 #endif
```

Index

/home/student/Desktop/Project/Automata-Libary/include/Automata.h,
13
/home/student/Desktop/Project/Automata-Libary/include/CFG.h,
13
/home/student/Desktop/Project/Automata-Libary/include/DFA.h,
14, 15
/home/student/Desktop/Project/Automata-Libary/include/NFA.h,
18, 19
/home/student/Desktop/Project/Automata-Libary/include/PDA.h,
23
/home/student/Desktop/Project/Automata-Libary/include/Regex.h,
23
/home/student/Desktop/Project/Automata-Libary/include/Regular_Language.h,
25
/home/student/Desktop/Project/Automata-Libary/include/STL_util.h,
26

Automata, 7

CFG< V_t, S_t >, 7

DFA< Q_t, S_t >, 8

epsilon, 8

NFA< Q_t, S_t >, 10

Parse_Tree, 11
Parse_Tree_Node, 11
PDA, 12

RegEx, 12

std::hash< std::list< T > >, 8
std::hash< std::pair< S, T > >, 9
std::hash< std::set< S > >, 9
std::hash< std::unordered_map< S, T, H > >, 9
std::hash< std::unordered_set< T > >, 9
std::hash< std::vector< T > >, 10